

Evaluación del Desempeño del Algoritmo de Sentimientos

Este cuaderno está diseñado para ejecutar y mostrar gráficamente el resultado de **4 métricas de alto nivel** propuestas por un Científico de Datos para evaluar el desempeño del modelo (RoBERTuito).

Las 4 Métricas Propuestas:

1. **Matriz de Confusión (Confusion Matrix):** Evalúa dónde exactamente se confunde el modelo (ej. ¿cuántos "positivos" predijo como "neutros"?).
2. **Reporte de Clasificación (F1-Score, Precisión y Exhaustividad):** Métricas que ponderan el desempeño considerando posibles desbalances en las categorías.
3. **Distribución de Nivel de Confianza (Probabilidad):** Una métrica no supervisada para medir qué tan "seguro" está el modelo de sus decisiones.
4. **Curvas ROC Multiclase y AUC:** Mide la capacidad del modelo para distinguir eficientemente entre las distintas clases (Positivo, Negativo, Neutro).

IMPORTANTE SOBRE LOS DATOS: Dado que nuestra base de datos `base_limpia.db` actual solo tiene las predicciones de la IA y NO cuenta con etiquetas establecidas por un humano (Ground Truth), en este cuaderno **simularemos** una columna `sentimiento_real` perturbando levemente la predicción original para poder realizar la demostración funcional y gráfica de las métricas. Una vez que etiquetes datos manualmente, solo debes reemplazar esa columna simulada por tus datos reales.

```
In [ ]: import sqlite3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report, roc_cur
from sklearn.preprocessing import label_binarize

import warnings
warnings.filterwarnings('ignore')

# Estilo para los gráficos
plt.style.use('ggplot')
sns.set_theme(style="whitegrid")

# 1. Conectar a la base de datos limpia y cargar datos
conn = sqlite3.connect('../data/base_limpia.db')
query = "SELECT texto_limpio, sentimiento, probabilidad FROM comentarios WHE
```

```

df = pd.read_sql_query(query, conn)
conn.close()

# 2. SIMULAR etiquetas reales (Ground Truth) para poder correr las métricas
np.random.seed(42) # Para reproducibilidad

def simular_error(pred):
    # Simulamos que el modelo acierta el 85% de las veces, para tener una gr
    if np.random.rand() > 0.85:
        opciones = [s for s in ['positivo', 'negativo', 'neutro'] if s != pr
        return np.random.choice(opciones)
    return pred

df['sentimiento_real'] = df['sentimiento'].apply(simular_error)
print(f"Total de registros cargados: {len(df)}")
df.head(3)

```

1. Matriz de Confusión

```

In [ ]: etiquetas = ['positivo', 'negativo', 'neutro']
y_real = df['sentimiento_real']
y_pred = df['sentimiento']

cm = confusion_matrix(y_real, y_pred, labels=etiquetas)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=etiquetas, yticklabels=etiquetas)
plt.title('Matriz de Confusión', fontsize=16)
plt.ylabel('Sentimiento Real (Ground Truth)')
plt.xlabel('Sentimiento Predicho (Modelo IA)')
plt.show()

```

2. Reporte de Clasificación (F1-score, Precision, Recall)

```

In [ ]: print("=== REPORTE DE CLASIFICACIÓN ===")
reporte = classification_report(y_real, y_pred, target_names=etiquetas)
print(reporte)

```

3. Distribución de Nivel de Confianza

Esta métrica analiza qué tan segura está la IA de sus respuestas en cada clase. Idealmente, queremos que la curva se concentre hacia la derecha (valores cercanos a 1.0).

```

In [ ]: plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='probabilidad', hue='sentimiento', kde=True, bins=30)
plt.title('Distribución de Probabilidad/Confianza del Modelo por Clase', fontweight='bold')
plt.xlabel('Probabilidad de Predicción')
plt.ylabel('Cantidad de Comentarios')
plt.show()

```

4. Curvas ROC Multiclase y AUC

Evalúa a través de diferentes umbrales qué tan separables son las clases. (Para este cálculo multiclase binarizamos las etiquetas).

```
In [ ]: # Binarizar las etiquetas
y_real_bin = label_binarize(y_real, classes=etiquetas)
y_pred_bin = label_binarize(y_pred, classes=etiquetas)
n_classes = y_real_bin.shape[1]

plt.figure(figsize=(9, 7))
colores = ['green', 'red', 'gray']

for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_real_bin[:, i], y_pred_bin[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=colores[i], lw=2,
             label=f'ROC {etiquetas[i]} (AUC = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Tasa de Falsos Positivos (FPR)')
plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
plt.title('Curvas ROC Multiclase', fontsize=16)
plt.legend(loc="lower right")
plt.show()
```